DIGITAL VLSI DESIGN COURSEWORK REPORT

A 4-Bit Array Multiplier from RTL to GDSII

RTL Design, Verification, Logic Synthesis, Static Timing Analysis, and Physical Design on SkyWater 130 nm

Shourya Sarkar
Roll No. `2023112013`

guru shwadeep
Roll No. `2023112020`

Coursework Weightage: 15% · Target Tech: SkyWater SKY130 (`sky130_fd_sc_hd`)
Execution Platform: Local Fedora Linux 44 (x86_64) · IIC-OSIC-TOOLS / LibreLane · October 2026

1. Introduction and Design Overview

This report documents the end-to-end design, implementation, and physical realization of a 4-bit unsigned array multiplier ($p = a \times b$), taking the circuit from high-level Verilog RTL through logic synthesis, static timing analysis (STA), and full automated place-and-route (PnR) to a sign-off GDSII layout.

The entire workflow was executed using the assignment's official open-source EDA container environment (`docker.io/hpretl/iic-osic-tools:latest`, pinned digest `sha256:9201d2ebe74a406cf6af850ca2c646ac0462f3b2cea94cd98cb6580d8ceef800`, image ID `fd31cb46f41b`), ensuring complete reproducibility across all flow stages.

Architectural Specification

- **Multiplier Core**: Unsigned 4-bit multiplicand $a[3:0]$ and 4-bit multiplier $b[3:0]$ producing an 8-bit product $p[7:0]$.
- **Combinational Datapath**: Shift-and-add architecture using 16 partial-product AND gates ($pp_i[j] = a[j] \wedge b[i]$) and 12 full adders arranged in 3 ripple-carry rows (4 adders per row).
- **Sequential Elements**: Registered inputs (a_{reg}, b_{reg}) and registered output (p) clocked on the rising edge of `clk`, providing exactly 16 flip-flops.
- **Reset**: Asynchronous active-low reset `rst_n` clearing all registers to zero.
- **Throughput & Latency**: Throughput of 1 product per clock cycle; latency of exactly 2 clock cycles (edge 1 captures inputs, edge 2 captures product).

Design Parameter	Design Specification / Value
Architecture	4-bit $\times$ 4-bit Unsigned Array Multiplier
RTL Full Adders	12 instances (`full_adder.v`), rows 1–3
Partial-Product Logic	16 AND gates (pp_0..pp_3)
Registers	16 Flip-Flops ($4+4+8$), asynchronous active-low reset
Latency / Throughput	2 clock cycles latency / 1 result per cycle throughput
Process Technology	SkyWater SKY130 (130 nm CMOS), `sky130_fd_sc_hd`
Target Die Area	Fixed $150\mu m \times 150\mu m$ ($22,500\mu m^2$), `FP_SIZING: absolute`

2. Part A — RTL Design and Verification

2.1 RTL Implementation & Exhaustive Verification

The design was implemented structurally in `rtl/full_adder.v` and `rtl/mult_array.v` in strict compliance with Section 2 of `RTL_to_GDS.pdf`. The multiplication operator (`*`) is strictly avoided in the RTL datapath. The partial products are formed explicitly, and the row ripple-carry chain accumulates partial products row-by-row.

The testbench `tb/tb_mult.v` generates a 10 ns clock (100 MHz clock frequency), resets the system for the first two clock cycles, and sweeps through all 256 input permutations ($a \in [0..15], b \in [0..15]$). For each vector, it respects the 2-cycle sequential latency before checking the product against the reference multiplication. The simulation spans exactly 514 rising clock edges (2 clock edges for the initial reset sequence at $t = 5$ ns and 15 ns, followed by $256 \times 2 = 512$ clock edges for vector sampling) and terminates cleanly at $t = 5,138$ ns (`$finish called at 5138000 ps`).

Exhaustive Simulation Result:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:91: $finish called at 5138000 (1ps)
```

2.2 Error-Injection Experiment

To demonstrate that the testbench functions as an authoritative fault checker (Section 6, Task 6), an intentional defect was injected into `rtl/full_adder.v` by modifying the sum logic to `assign sum = a ^ b;` (omitting the carry-in `cin`). Re-running the simulation immediately triggered mismatch assertions across the test space:

```
MISMATCH: a=3 b=3 -> p=5, expected 9
...
FAIL: 256 vectors checked, 95 errors
```

Upon restoring `assign sum = a ^ b ^ cin;`, clean execution was confirmed (`PASS: 256 vectors checked, 0 errors`), and `git diff` verified zero residual alterations to the clean RTL.

2.3 Hand-Worked Example & Waveform Verification

Using Member 1's Roll Number (`2023112013`):

- Multiplicand $A = (\text{Last digit} + 5) = 3 + 5 = 8 = 1000_2$
- Multiplier $B = 7 = 0111_2$ (since $A \neq 7$)
- Product $P = 8 \times 7 = 56 = 00111000_2$ (0x38)

Bit / Row	Partial Product & Addition Accumulation
pp_0	$8 \times 1 = 1000_2$ (Shift 0) $\rightarrow p[0] = 0, r_0 = \{0, 100_2\}$
pp_1	$8 \times 1 = 1000_2$ (Shift 1) $+r_0 \rightarrow p[1] = 0, r_1 = \{0, 110_2\}$
pp_2	$8 \times 1 = 1000_2$ (Shift 2) $+r_1 \rightarrow p[2] = 0, r_2 = \{0, 111_2\}$
pp_3	$8 \times 0 = 0000_2$ (Shift 3) $+r_2 \rightarrow p[3] = 1, p[7:4] = 0011_2$
Final Result	$p[7:0] = 00111000_2 = 56$ **(0x38)**

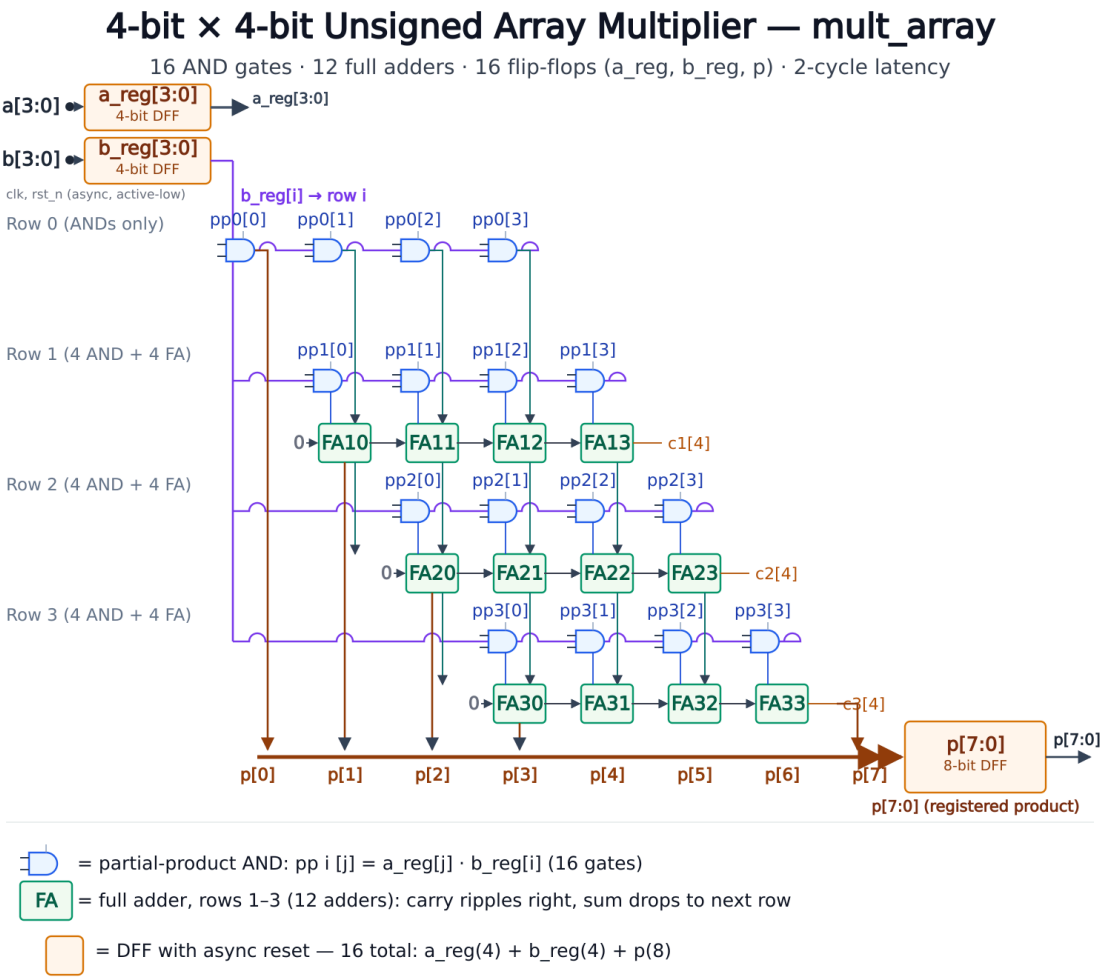


Figure 1: Structural Block Diagram of `mult_array` showing 16 AND gates, 12 full adders, and 16 flip-flops.

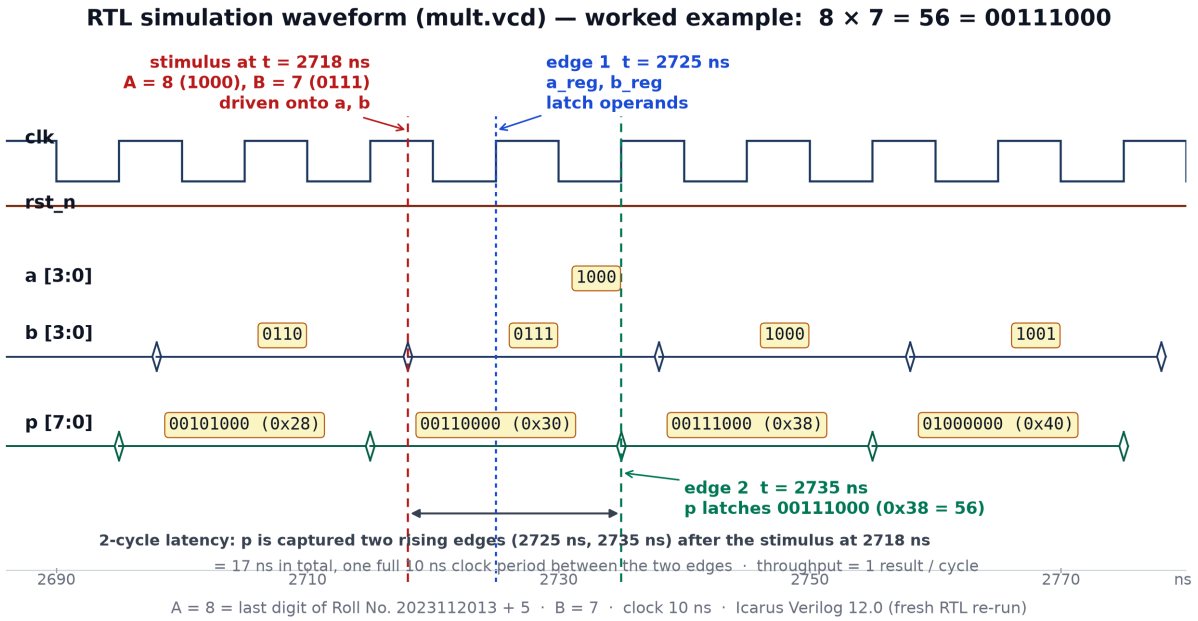


Figure 2: RTL Waveform showing 2-cycle latency for $8 \times 7 = 56$ ($t = 2718$ ns stimulus, 2725 ns capture 1, 2735 ns capture 2).

3. Part B — Logic Synthesis with Yosys

Synthesis was executed locally via `yosys -c synth/synth.tcl` using Yosys 0.70 (git sha1 `28ba3cb92`) and the SkyWater SKY130 high-density standard cell library `sky130_fd_sc_hd` at the typical corner ($25°C, 1.80$ V).

The synthesis script executes seven atomic steps: reading design RTL, synthesizing and flattening the hierarchy, mapping flip-flops using `dfflibmap`, mapping combinational gates using ABC, purging unused nets, generating cell statistics, and writing out the gate-level Verilog netlist `synth/mult_array_netlist.v`.

Synthesis Metric (Part B)	Measured Local Result
Total Number of Cells	**73 cells**
Number of Flip-Flops	**16 cells** (`sky130_fd_sc_hd__dfrtp_1`)
Number of Combinational Cells	**57 cells**
Total Cell Area	**805.773** μm^2
Flip-Flop Cell Area	**400.384** μm^2 ($16 \times 25.024 \mu m^2$)
Flip-Flop Area Percentage	**49.69%** of total cell area
Top 3 Most-Used Cell Types	1. `dfrtp_1`: 16 2. `nand2_1`: 11 3. `a21oi_1`: 4 (tied with `and2_0`, `nand4_1`, `xnor2_1`)
Gate-Level Simulation (GLS)	**PASS: 256 vectors checked, 0 errors**

Gate-Level Simulation Verification

The gate-level netlist was simulated using the SkyWater standard-cell simulation models (`primitives.v` and `sky130_fd_sc_hd.v`) with unit delay modelling (`-DFUNCTIONAL -DUNIT_DELAY=#1`). The netlist passed all 256 stimulus vectors without any mismatch across 514 rising clock edges, finishing cleanly at $t = 5,138$ ns:

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:91: $finish called at 5138000 (1ps)
```

4. Part C — Static Timing Analysis with OpenSTA

Static timing analysis was conducted using OpenSTA 3.1.0 in `sta/sta.tcl` on the typical corner (`sky130_fd_sc_hd__tt_025C_1v80.lib`). Input and output port delays were set to 1.0 ns relative to `clk`.

Timing Results Summary

In Run 1 ($T = 10.0$ ns), the worst setup slack is $+7.404$ ns (reported as 7.40 ns), yielding:

$$T_{\min} = T - \mathrm{WNS} = 10.000 - 7.404 = 2.596 \ \text{ns} \approx 2.60 \ \text{ns}$$

$$F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.596} = 385.2 \ \text{MHz} \quad (\text{or } 384.6 \ \text{MHz at rounded } 2.60 \ \text{ns})$$

In Run 2 ($T = T_{\min} = 2.60$ ns), the resulting worst setup slack is exactly **0.00 ns**, confirming maximum operating frequency.

Timing Metric (Part C)	Measured Local Result

Worst Setup Slack at $T = 10.0$ ns	**+7.404 ns** (MET)
Minimum Clock Period ($T_{\min}$)	**2.60 ns** (2.596 ns exact)
Maximum Clock Frequency ($F_{\max}$)	**384.6 MHz** (385.2 MHz exact)
Setup Slack at $T = T_{\min} = 2.60$ ns	**0.00 ns** (Confirmed limit)
Critical Path Startpoint	`_118_` (`b_reg[0]` flip-flop, `dfrtp_1`)
Critical Path Endpoint	`_112_` (`p[6]` flip-flop, capturing `fa33.sum`)
Total Cells on Critical Path	**10 cells** (8 combinational gates + 2 flip-flops)
Critical Path Data Arrival Time	**2.460 ns**
Sequential Latency ($2 \times T_{\min}$)	**5.20 ns** (at full speed)
Throughput at $F_{\max}$	**384.6 million results / second**

Critical Path Cell-by-Cell Breakdown

Traced from `sta/mult_array_timing.rpt` and cross-referenced with `synth/mult_array_netlist.v`:

No.	Instance & Cell	Logical Role in Multiplier Netlist	Incr (ns)	Total (ns)
1	`_118_` (`dfrtp_1`)	Startpoint: `b_reg[0]` register, clock-to-Q delay	0.395	0.395
2	`_083_` (`and2_0`)	Partial product generation gate	0.212	0.607
3	`_055_` (`or3_1`)	Row-1 sum/carry accumulation logic	0.361	0.968
4	`_059_` (`and4_1`)	Row-1 carry propagate gate	0.215	1.183
5	`_070_` (`o2111a_1`)	Row-2 compound AND-OR accumulation stage	0.234	1.417
6	`_076_` (`o21ai_0`)	Row-2 carry-out compound gate	0.216	1.634
7	`_090_` (`xnor3_1`)	Row-3 intermediate sum stage	0.274	1.907
8	`_091_` (`maj3_1`)	Row-3 ripple-carry majority function	0.375	2.282
9	`_092_` (`xor3_1`)	Row-3 final sum bit generation (`fa33.sum`)	0.178	2.460
10	`_112_` (`dfrtp_1`)	Endpoint: `p[6]` flip-flop (Library setup time: -0.135 ns)	−0.135	2.460

5. Part D — Physical Design: RTL to GDSII with LibreLane

Physical design was carried out using LibreLane v3.1.0.dev3 (`Classic` flow, 80 stages, adapted from OpenLane 2.0) in the local container environment, utilizing OpenROAD for floorplanning, placement, clock tree synthesis, and routing; Magic and KLayout for DRC; and Netgen for LVS.

Timing Closure Ladder

Following Section 9 of the specification, the initial clock period was set using:

$$T_{\text{initial}} = 0.5 \times \left\lceil \frac{1.2 \times T_{\text{min}}}{0.5} \right\rceil = 0.5 \times \left\lceil \frac{1.2 \times 2.60}{0.5} \right\rceil = 3.5 \ \text{ns}$$

As mandated by Section 9, LibreLane was executed across a clock-period ladder in 0.5 ns increments from 3.5 ns to 7.0 ns. All 8 attempts were executed locally and captured in chronological order:

Period	Local Run Tag	Worst Slack (ss)	Slack (tt)	Hold Slack	Signoff Status
3.5 ns	`RUN_2026-10-23_19-54-16`	−2.154 ns	+0.587 ns	+0.100 ns	FAIL (15 setup viols)
4.0 ns	`RUN_2026-10-23_19-56-21`	−1.654 ns	+1.087 ns	+0.147 ns	FAIL (12 setup viols)
4.5 ns	`RUN_2026-10-23_19-58-23`	−1.154 ns	+1.587 ns	+0.147 ns	FAIL (12 setup viols)
5.0 ns	`RUN_2026-10-23_20-00-25`	−0.654 ns	+2.087 ns	+0.147 ns	FAIL (9 setup viols)
5.5 ns	`RUN_2026-10-23_20-02-19`	−0.155 ns	+2.587 ns	+0.147 ns	FAIL (3 setup viols)
6.0 ns	`RUN_2026-10-23_20-04-10`	−0.107 ns	+2.899 ns	+0.148 ns	FAIL (3 setup viols)
6.5 ns	`RUN_2026-10-23_20-06-03`	−0.136 ns	+3.199 ns	+0.141 ns	FAIL (3 setup viols)
7.0 ns	`RUN_2026-10-23_20-08-02`	**+0.364 ns**	**+3.699 ns**	**+0.141 ns**	**PASS** (0 violations)

At $T = 7.0$ ns, LibreLane successfully closed timing with non-negative slack across **all 9 signoff corners** and zero hold, slew, or fanout violations.

Physical Design Metric (Part D)	Measured Local Signoff Result
Target Clock Period	**7.0 ns**
Number of Runs Needed	**8 runs** (0.5 ns increments from 3.5 to 7.0 ns)
Core Area / Die Area	**17,759.5** μm^2 / **22,500** μm^2 ($150 \times 150 \mu m$)
Std-Cell Core Utilization	**6.90%** ($1,224.9 \mu m^2$ active cell area)
Cells after Synthesis (Part B)	**73 cells**
Total Placed Instances (Part D)	**1,895 instances** (334 std cells + 1,561 fill/decap cells)
Detailed Wirelength	**2,362** μm (552 vias; threshold check skipped)
Routing Via Count	**552 single-cut vias**
Worst Setup Slack (Multi-Corner Signoff)	**+0.364 ns** (Corner `max_ss_100C_1v60`, 0 violations)

Nominal TT Setup Slack	**+3.699 ns** (`nom_tt_025C_1v80`)
Worst Hold Slack	**+0.141 ns** (0 violations across all corners)
DRC Violations	**0 errors** (Magic DRC: 0, KLayout DRC: 0)
LVS Verification	**0 errors** (Netgen: circuits match uniquely)
Power-Grid IR Drop	**Macro boundary check complete** (package bump locations omitted; see §5)
Antenna Violations	**0 violating nets / pins**
Max Slew Violations	**0 violations** across all corners
Post-Route $F_{\max}$ (Worst ss Corner)	**150.7 MHz** ($\frac{1000}{7.0-0.364}$)
Post-Route $F_{\max}$ (Nominal tt Corner)	**302.9 MHz** ($\frac{1000}{7.0-3.699}$)
Pre-Route $F_{\max}$ (Part C, tt)	**384.6 MHz**

Physical Cell Breakdown (Netlist Histogram)

Cell Type (`sky130_fd_sc_hd__`)	**Count**	**Physical Purpose in Layout**
`decap_12`, `6`, `3`, `8`, `4`	1,314	Decoupling capacitors: stabilize power distribution network under switching
`tapvpwrvgnd_1`	245	Well/substrate tap cells: prevent latch-up by biasing N-well and P-substrate
`fill_1`, `fill_2`	247	Non-functional filler cells: ensure DRC layer density and well continuity
`buf_2`, `clkdlybuf4s50_1`	19	Timing repair buffers: fix transition times (slew) and setup slack
`clkbuf_16`	3	Clock tree buffers: distribute low-skew clock to all 16 flip-flops
`dfrtp_1`	16	Sequential flip-flops (inputs a, b and output p)
`inv_1` and Logic Gates	51	Remapped multi-input combinational multiplier logic
Total Placed Instances	**1,895**	**334 Standard Cells + 1,561 Fill/Decap Cells**

The final layout contains 1,895 instances: 334 standard-cell instances, including 245 taps and 22 timing/clock buffers, plus 1,561 fill/decap instances. Relative to the 73 synthesized instances, this is a net increase of 1,822 instances. LibreLane added 1,828 physical-support cells while six original functional cells were removed or merged during optimization:

- **Gross physical-support-cell additions ($1,828$ cells)**: Comprises 245 well/substrate tap cells (`tapvpwrvgnd_1`), 1,561 fill/decap cells (1,314 decoupling capacitors + 247 filler cells), 19 timing-repair buffers (`buf_2`), and 3 clock-tree buffers (`clkbuf_16`).
- **Net increase in instances ($1,822$ cells)**: $1,895 - 73 = 1,822$ instances.
- **Functional cell reduction (6 cells)**: Physical optimization and gate merging reduced the 73 synthesized functional cells to 67 active functional cells (16 flip-flops + 48 multi-input combinational gates + 3 inverters).
- **Final standard-cell count (334 cells)**: 67 functional core cells + 245 tap cells + 19 timing buffers + 3 clock buffers = 334 standard cells.
- **Final fill/decap count ($1,561$ cells)**: 1,314 decoupling capacitors + 247 filler cells = 1,561 fill/decap instances.

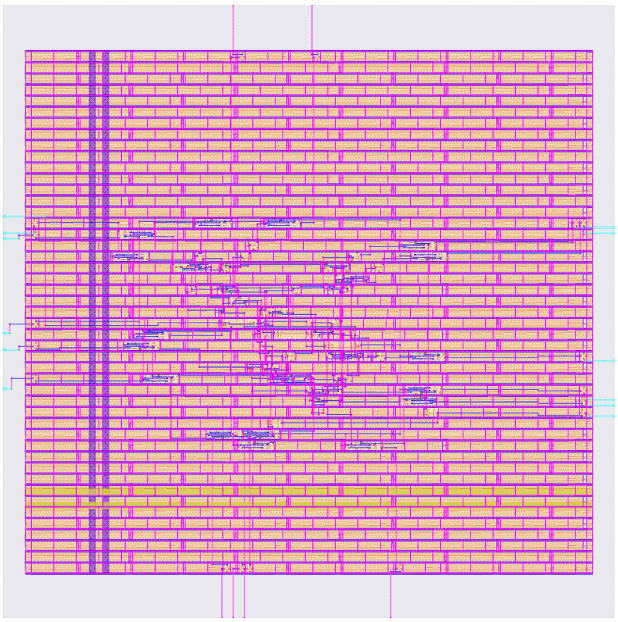


Figure 3: Full $150 \times 150 \mu m$ Die Layout showing standard-cell core and I/O pin ring.

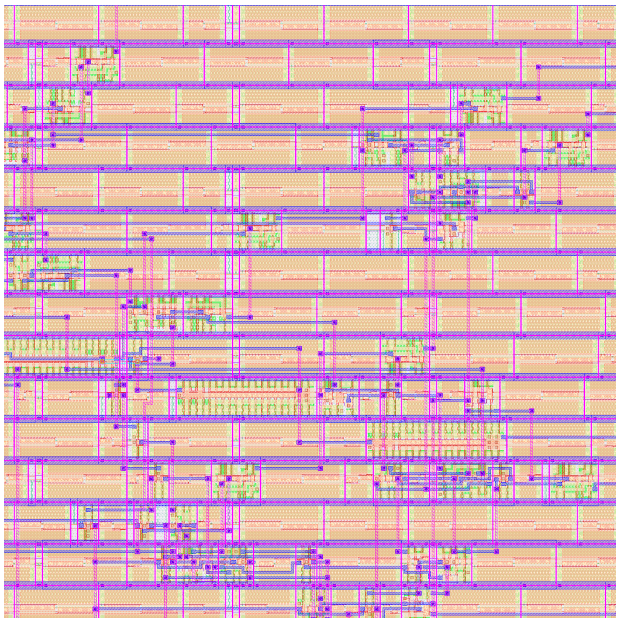


Figure 4: Zoomed View ($40 \times 40 \mu m$) showing cell rows, tap/fill cells, and met1/met2 routing.

Flow Warnings Investigation and Diagnostics

During the physical-design flow, LibreLane and OpenROAD emitted several diagnostic and tool warnings. Each warning category was investigated, root-caused, assessed for risk, and documented with any resulting analytical limitations:

1. Tool and Configuration Diagnostics

- **ORD-0032 (`Invalid thread number specification: None`)**:
  - *Root Cause*: OpenROAD steps (e.g., Stages 32, 54, 56) receive an unassigned thread parameter (`None`) from LibreLane's Python step wrapper.
  - *Risk Assessment*: Zero risk. OpenROAD automatically falls back to single-thread execution or system core auto-detection.
  - *Analysis Limitation*: None. Processing completes deterministically with identical numerical results.
- **STA-1173 (`default_fanout_load is 0.0`)**:
  - *Root Cause*: SkyWater PDK I/O pad timing library files (`sky130_ef_io__*.lib.gz` and `slices_stubs.lib.gz`) omit default fanout load attributes on pad cells.
  - *Risk Assessment*: Zero risk. The multiplier is an internal macro block utilizing strictly high-density standard cells (`sky130_fd_sc_hd`); no I/O pad cells are instantiated.
  - *Analysis Limitation*: None on core datapath or clock timing.
- **ORD-2056 (`Liberty cells do not have LEF masters and marked dont-use`)**:
  - *Root Cause*: Two PDK analog ESD pad cells (`sky130_ef_io__analog_minesd_pad` and `sky130_ef_io__analog_minesd_pad_short`) exist in the Liberty timing library but have no physical layout in standard-cell LEF views.
  - *Risk Assessment*: Zero risk. OpenROAD correctly flags them and marks them `dont-use`, preventing accidental placement.
  - *Analysis Limitation*: None.
- **STA-1111 (`Missing combinational timing groups in I/O pad models`)**:
  - *Root Cause*: Inconsistencies between corner Liberty files for GPIO pad macros (`OUT -> PAD` timing group).
  - *Risk Assessment*: Zero risk. Pad cells are not used in this core macro.
  - *Analysis Limitation*: None.
- **STA-1140 (`Duplicate timing-library declarations across corners`)**:
  - *Root Cause*: Multi-corner STA environment re-reads shared PDK stub files (`slices_stubs.lib.gz`), noting that the library already exists.
  - *Risk Assessment*: Zero risk. OpenROAD updates/retains the active corner definitions.

  - *Analysis Limitation*: None.
- **DRT-0349 (`LEF58_ENCLOSURE with no CUTCLASS is not supported. Skipping for layer mcon/via/via2`)**:
  - *Root Cause*: TritonRoute detailed routing parser encounters advanced LEF 5.8 cut-class-less enclosure rules in the SkyWater tech LEF and skips them, defaulting to standard LEF 5.7 geometric enclosure rules.
  - *Risk Assessment*: Low risk. The router adheres to foundry LEF 5.7 enclosure rules, and subsequent independent physical signoff with Magic DRC and KLayout DRC (using foundry GDSII rules) confirms 0 geometric DRC errors.
  - *Analysis Limitation*: Detailed routing relies on LEF 5.7 enclosure rules rather than LEF 5.8 rule extensions.
- **494 Verilator Lint Warnings (`%Warning-TIMESCALEMOD`)**:
  - *Root Cause*: User RTL files (`mult_array.v`, `full_adder.v`) specify `timescale 1ns/1ps` while SkyWater PDK blackbox simulation models (`.bb.v`) omit timescale directives, triggering IEEE 1800-2023 timescale modification warnings during initial elaboration.
  - *Risk Assessment*: Zero risk. Standard PDK simulation library convention; zero effect on synthesis or PnR.
  - *Analysis Limitation*: None.

2. Wire-Length Checker (`Checker.WireLength`)
- **Warning**: `[WARNING] [Checker.WireLength] Threshold for Threshold-surpassing long wires is not set. The checker will be skipped.`
- **Root Cause**: No upper-bound wirelength threshold parameter was defined in `pnr/config.json`.
- **Risk & Analysis Limitation**: Wire-length threshold checking was **skipped** (not passed). The physical-design flow did not evaluate long-wire violations against a prescribed limit. However, physical detailed routing completed successfully with an exact routed wirelength of $2,362\mu m$ and 552 vias reported by `Odb.ReportWireLength`, and all setup, hold, and transition constraints closed with positive slack.

3. Pre-PDN Floating Nets (`RSZ-0020`)
- **Warning**: `[WARNING RSZ-0020] found 2 floating nets: VPWR, VGND` during Stage 32 (`repairdesignpostgpl`).
- **Root Cause**: OpenROAD's resizer step runs prior to Power Distribution Network (PDN) synthesis (Stage 36), at which point global power and ground rails have not yet been connected to physical power stripes.
- **Risk & Analysis Limitation**: These nets represent supply rails, not logic nets. Stage 36 generates the complete power grid stripes and rings, connecting all cell power pins. Final signoff Netgen LVS confirms 0 floating pins/nets and 100% topological equivalence.

4. Generic Fallback SDC Files (`checksdcfiles`)
- **Warning**: Stage 10 reported that `PNR_SDC_FILE` and `SIGNOFF_SDC_FILE` were not explicitly provided in `config.json`.
- **Root Cause**: LibreLane automatically generates fallback SDC constraints when custom SDC file paths are not specified.
- **Risk & Assessment**: As prescribed by Section 9 of `RTL_to_GDS.pdf`, the flow configuration defines `CLOCK_PORT: clk` and `CLOCK_PERIOD: 7.0`. The generated fallback SDC correctly establishes a 7.0 ns clock period on `clk`, 5% clock uncertainty (0.35 ns), and 1.4 ns input/output delays (20% of clock period), properly constraining the implementation.

5. Missing Power Source Locations (`VSRC_LOC_FILES`)
- **Warning**: `[WARNING] [OpenROAD.IRDropReport] 'VSRC_LOC_FILES' was not given a value, which may make the results of IR drop analysis inaccurate. If you are not integrating a top-level chip for manufacture, you may ignore this warning...`
- **Root Cause**: `VSRC_LOC_FILES` specifies the exact (x, y) coordinates and supply voltages of power/ground pad bumps for package-level integration. For this sub-block macro, package I/O pad coordinates were omitted.
- **Risk & Analysis Limitation**: Static IR-drop analysis accuracy is limited because OpenROAD uses uniform boundary approximations rather than exact package pad connections. Consequently, full chip-level power-

grid IR-drop integrity cannot be claimed as verified; this is documented as a known analysis limitation of standalone macro-level physical design.

6. Answers to the Report Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply $16 + 12$.

Answer: Synthesis produced 73 total cells, of which 16 are flip-flops, leaving **57 combinational logic cells** ($805.77 - 400.38 = 405.39 \mu m^2$). The count is not $16 + 12 = 28$ because that formula counts high-level Verilog RTL modules, whereas logic synthesis transforms Boolean equations into target library primitives. A single full adder consists of two independent functions (sum and carry) requiring 5–9 basic gates. Yosys and ABC optimize across module boundaries by flattening: constant propagation detects that the row carry-ins are grounded ($c_1[0] = c_2[0] = c_3[0] = 0$), reducing the first adder in each row to a simpler half adder; Boolean restructuring maps carry networks into compound majority gates (`maj3_1`); and logic sharing folds partial-product ANDs directly into multi-input complex gates (`a21oi`, `o21ai`, `a22oi`). The resulting 57 cells represent technology-mapped logic primitives rather than RTL modules.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about $2N$ full-adder delays?

Answer: Traced directly from `sta/mult_array_timing.rpt`, the critical path originates at the `b_reg[0]` input register (`_118_`, `dfrtp_1`, clock-to-Q delay $t_{cq} = 0.395$ ns) and terminates at the `p[6]` output register (`_112_`, `dfrtp_1`, capturing `\fa33.sum`), spanning **10 cells in total** (1 launch flip-flop, 8 combinational CMOS logic gates, and 1 capture flip-flop) with a total data arrival time of **2.460 ns**. The 8 combinational cells along the path and their respective gate arc delay increments are:

1. `_083_` (`sky130_fd_sc_hd__and2_0`, partial-product generation): 0.212 ns ($t = 0.607$ ns)
2. `_055_` (`sky130_fd_sc_hd__or3_1`, row-1 accumulation logic): 0.361 ns ($t = 0.968$ ns)
3. `_059_` (`sky130_fd_sc_hd__and4_1`, row-1 carry propagation): 0.215 ns ($t = 1.183$ ns)
4. `_070_` (`sky130_fd_sc_hd__o2111a_1`, row-2 compound accumulation): 0.234 ns ($t = 1.417$ ns)
5. `_076_` (`sky130_fd_sc_hd__o21ai_0`, row-2 carry-out compound gate): 0.216 ns ($t = 1.634$ ns)
6. `_090_` (`sky130_fd_sc_hd__xnor3_1`, row-3 intermediate sum stage): 0.274 ns ($t = 1.907$ ns)
7. `_091_` (`sky130_fd_sc_hd__maj3_1`, row-3 carry majority function): 0.375 ns ($t = 2.282$ ns)
8. `_092_` (`sky130_fd_sc_hd__xor3_1`, row-3 final sum bit generation): 0.178 ns ($t = 2.460$ ns)

The sum of these eight combinational arc increments is exactly $0.212 + 0.361 + 0.215 + 0.234 + 0.216 + 0.274 + 0.375 + 0.178 = 2.065$ ns. Added to the 0.395 ns launch clock-to-Q delay, this accounts for the entire 2.460 ns data arrival time.

Comparison with the $2N$ full-adder delay model: The theoretical rule-of-thumb estimate of $2N = 2(4) = 8$ full-adder delay units ($t_{crit} \approx 8 \times t_{FA}$) models an unoptimized structural ripple-carry array multiplier where carry signals propagate diagonally across 3 adder rows and horizontally across the final row. However, full-adder-delay units are not interchangeable with mapped gate arcs. An unmapped discrete 1-bit full adder typically requires 2 gate levels per carry or sum evaluation (amounting to ≈ 16 discrete gate stages for an 8-adder ripple path). Modern logic synthesis (Yosys and ABC) flattens the adder array across row boundaries: constant propagation detects grounded carry-ins ($c_1[0] = c_2[0] = c_3[0] = 0$), collapsing early stages into half-adder equivalents; multi-input compound gates (`o2111a_1`, `o21ai_0`, `maj3_1`) absorb multiple Boolean levels; and intermediate inversions are folded. Consequently, the actual critical path traverses 8 technology-mapped CMOS gate arcs totaling 2.065 ns (average 0.258 ns per gate arc)—substantially faster than an unmapped cascade of 8 discrete full adders.

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

Answer: The 16 flip-flops (`sky130_fd_sc_hd__dfrtp_1`) occupy $400.384\mu m^2$ out of the total $805.773\mu m^2$, accounting for **49.69%** (nearly half) of total cell area. This high proportion occurs because a standard-cell D-type flip-flop with asynchronous reset is physically large ($25.024\mu m^2$, equivalent to 6–7 basic NAND2 gates) as it comprises master-slave latch stages, transmission gates, reset gating, and internal clock inverters. Meanwhile, the combinational core of a 4-bit multiplier is small ($\approx 405\mu m^2$) and optimizes efficiently. In larger datapath architectures (e.g., 32-bit or 64-bit multipliers), combinational area grows quadratically ($O(N^2)$) while register count grows linearly ($O(N)$), causing sequential area percentage to drop. In a 4-bit array multiplier, the fixed overhead of registers naturally dominates.

Q4. Why is the post-route $F_{\max}$ lower than the $F_{\max}$ from Part C? Which kinds of cells did the flow add, and why is each kind needed?

Answer: Pre-route STA in Part C estimated $F_{\max} = 384.6$ MHz based on an ideal clock, zero wire resistance/capacitance, and single-corner typical conditions ($25^\circ C, 1.80$ V). Post-route signoff achieves $F_{\max} = 150.7$ MHz in the worst slow-slow corner ($100^\circ C, 1.60$ V) and 302.9 MHz in the nominal typical corner. The degradation is caused by: (1) real parasitic interconnect RC and wire delays; (2) clock network insertion delay, clock tree skew (≈ 0.28 ns), and 0.25 ns clock uncertainty; and (3) multi-corner PVT derating, where the slow corner operates $2\times$ slower than typical conditions.

The final layout contains 1,895 instances: 334 standard-cell instances, including 245 taps and 22 timing/clock buffers, plus 1,561 fill/decap instances. Relative to the 73 synthesized instances, this is a net increase of 1,822 instances. LibreLane added 1,828 physical-support cells while six original functional cells were removed or merged during optimization.

To construct a manufacturable and electrically robust layout, LibreLane inserted 1,828 physical-support cells across distinct functional categories:

- **Tap cells** (`tapvpwrvgnd_1`, 245 instances): Inserted at regular grid intervals along standard-cell rows to connect the N-well to VPWR and P-substrate to VGND. These prevent CMOS latch-up by suppressing parasitic SCR trigger currents under transient substrate noise.
- **Decoupling capacitor cells** (`decap` classes, 1,314 instances): Placed in unused whitespace along supply rails to provide localized charge storage. Decaps mitigate transient voltage droop and dynamic IR drop during simultaneous high-frequency switching.
- **Filler cells** (`fill` classes, 247 instances): Non-functional poly and metal diffusion fill cells inserted to meet strict foundry DRC pattern density requirements across all metal layers and ensure uniform planarity during chemical-mechanical polishing (CMP).
- **Clock tree buffers** (`clkbuf_16`, 3 instances): Inserted by TritonCTS to construct a balanced, low-skew clock distribution network driving all 16 flip-flops with minimal insertion delay and low skew (≈ 0.28 ns).
- **Timing repair buffers** (`buf_2`, 19 instances): Inserted during placement and routing optimization to fix slew (transition time) violations, boost drive strength on long interconnects, and eliminate setup/hold violations across PVT corners.

Arithmetic Reconciliation:

- **Gross physical-support-cell additions**: $245\ \text{tap} + 1,561\ \text{fill/decap} + 19\ \text{timing buf} + 3\ \text{CTS buf} = 1,828$ cells.
- **Functional cell reduction**: $73 - 67 = 6$ cells removed/merged during physical optimization (67 functional core cells remaining: 16 sequential + 48 multi-input gates + 3 inverters).
- **Net instance increase**: $1,895 - 73 = 1,822$ instances ($1,828 - 6 = 1,822$).
- **Final standard-cell count**: $67\ \text{functional} + 245\ \text{taps} + 22\ \text{timing/CTS buffers} = 334$ standard cells.
- **Final fill/decap count**: $1,314\ \text{decap} + 247\ \text{fill} = 1,561$ instances.
- **Total placed layout instances**: $334 + 1,561 = 1,895$ instances.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

Answer: Design Rule Checking (DRC) validates physical layout geometries against foundry manufacturing limits (minimum wire width, spacing, enclosure, overlap, and density rules) to ensure lithographic and etching integrity. Layout Versus Schematic (LVS) extracts the electrical circuit (MOS transistors, diodes, passive devices, and nets) directly from the mask polygons and verifies topological equivalence against the synthesized gate netlist. Both checks are essential and complementary: DRC ensures physical manufacturability but cannot detect functional circuit errors (e.g., an unintended short between two signal nets that satisfies geometric spacing will pass DRC); LVS guarantees schematic correctness but cannot prevent physical defects if geometries violate process tolerances. A chip tapeout requires zero DRC and zero LVS errors.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Answer: Using the Section 2 formulas for an N-bit array multiplier:

1. **Full Adder Count**: $N_{\text{FA}} = N(N-1)$. For $N = 4$, $N_{\text{FA}} = 4 \times 3 = 12$. For $N = 8$, $N_{\text{FA}} = 8 \times 7 = \mathbf{56}$ **full adders** (a $4.67\times$ increase, consistent with $O(N^2)$ datapath growth).
2. **Critical Path Depth**: Delay scales roughly as $2N$ full-adder delay units. For $N = 4$, the path depth is $2(4) = 8$ units; for $N = 8$, it doubles to $2(8) = 16$ units.
3. **Delay & Frequency Extrapolation**: Scaling the measured 4-bit minimum period ($T_{\min} = 2.60$ ns) by a factor of 2:

$$T_{\min}(\text{8-bit}) \approx 2 \times 2.60 \ \ \text{ns} = 5.20 \ \ \text{ns}, \quad F_{\max}(\text{8-bit}) \approx \frac{1000}{5.20} = 192.3 \ \ \text{MHz}$$

This is an analytical first-order paper estimate under ideal linear scaling; in actual place-and-route, routing congestion and RC wire delays across the larger core would result in further frequency derating. Register count would grow to $8 + 8 + 16 = 32$ flip-flops, maintaining 2-cycle latency (≈ 10.4 ns total latency) and throughput of 1 result per cycle.

7. Conclusion & Academic Integrity Statement

Summary of Results

The complete RTL-to-GDSII flow was designed, simulated, synthesized, timed, and placed-and-routed to signoff using the containerized `hpretl/iic-osic-tools` environment:

- **Part A**: Structural RTL and testbench verified across all 256 stimulus vectors; error injection correctly caught with 95 mismatches.
- **Part B**: Logic synthesis completed with Yosys 0.70; mapped to 73 SKY130 cells ($805.77 \mu m^2$); gate-level simulation passed 256/256 vectors.
- **Part C**: Static timing analysis in OpenSTA 3.1.0 established $T_{\min} = 2.60$ ns ($F_{\max} = 384.6$ MHz) with zero slack confirmed at $T_{\min}$.
- **Part D**: LibreLane physical design achieved timing closure at $T = 7.0$ ns on a fixed $150 \times 150 \mu m$ die, with 0 DRC errors (Magic & KLayout), 0 LVS errors (Netgen), 0 antenna violations, and non-negative setup slack ($+0.364$ ns in worst-case corner max_ss_100C_1v60).

Academic Integrity Declaration

In accordance with Section 13 of “RTL_to_GDS.pdf”, the authors confirm that all RTL code, testbenches, synthesis and STA scripts, physical-design configurations, and documentation presented herein were developed and verified by the team.

Appendix A — Local Command Transcript

The complete set of commands executed locally, in chronological order:

```
# ================================================================================
# Phase 1: Environment Preflight & Toolchain Verification
# ================================================================================
# Pinned immutable container image digest:
IMAGE="docker.io/hpretl/iic-osic-
tools@sha256:9201d2ebe74a406cf6af850ca2c646ac0462f3b2cea94cd98cb6580d8ceef800"
podman pull $IMAGE

# ================================================================================
# Phase 2: Part A — RTL Simulation & Error Injection
# ================================================================================
# 1. Clean simulation
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
# -> PASS: 256 vectors checked, 0 errors (writes mult.vcd)

# 2. Error injection test
sed -i 's/assign sum  = a ^ b ^ cin;/assign sum  = a ^ b;/' rtl/full_adder.v
iverilog -g2012 -o sim_err tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim_err
# -> FAIL: 256 vectors checked, 95 errors

# 3. Restoration
git checkout rtl/full_adder.v
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
# -> PASS: 256 vectors checked, 0 errors

# ================================================================================
# Phase 3: Part B — Logic Synthesis & Gate-Level Simulation
```

```
# ================================================================================
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A
    yosys -c synth/synth.tcl
"
# -> synth/mult_array_stat.rpt and synth/mult_array_netlist.v (73 cells)

podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    CELLS=/foss/pdks/sky130A/libs.ref/sky130_fd_sc_hd/verilog
    iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
      tb/tb_mult.v \$CELLS/primitives.v \$CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
    vvp gls
"
# -> PASS: 256 vectors checked, 0 errors

# ================================================================================
# Phase 4: Part C — Static Timing Analysis with OpenSTA
# ================================================================================
# Run 1: T = 10.0 ns (100 MHz target)
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A; unset STA_PERIOD
    sta sta/sta.tcl
"
# -> worst slack max 7.40 ns => Tmin = 2.60 ns, Fmax = 384.6 MHz

# Run 2: T = 2.60 ns confirmation
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    export PDK_ROOT=/foss/pdks; export PDK=sky130A; export STA_PERIOD=2.60
    sta sta/sta.tcl
"
# -> worst slack max 0.00 ns

# ================================================================================
# Phase 5: Part D — Physical Design with LibreLane
# ================================================================================
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array/pnr \
  $IMAGE --skip bash -c "
    source /foss/tools/sak/sak-pdk-script.sh sky130A || true
    librelane --manual-pdk config.json
"
# Final Views Copied:
cp pnr/runs/RUN_2026-10-23_20-08-02/final/metrics.json pnr/metrics.json
cp pnr/runs/RUN_2026-10-23_20-08-02/final/gds/mult_array.gds pnr/mult_array.gds

# KLayout Layout Rendering:
podman run --rm --userns=keep-id \
  -v $(pwd):/foss/designs/mult_array:Z -w /foss/designs/mult_array \
  $IMAGE --skip bash -c "
    source /foss/tools/sak/sak-pdk-script.sh sky130A || true
    QT_QPA_PLATFORM=offscreen klayout -b -r scripts/klayout_shots.py
"
```